

Install, run, and inspect the simulation

A plain-English Windows + WSL guide for first-time users

This guide starts with an ordinary Windows computer and ends with a working Smoke test, an optional graphical view, a one-seed check, the full 450-run experiment, and result graphs.

WHAT YOU INSTALL	EXACT VERSION USED
WSL2 + opp_env	opp_env 0.36.1 family
OMNeT++	6.3.0
Veins	5.3.1
INET	4.6.0
SUMO	1.18.0

TWO DIFFERENT TERMINALS

PowerShell is the Windows terminal used only for WSL setup. After you open **opp_env**, every other command in this guide is typed in its Linux terminal. Copy one code box at a time. Do not type the box labels.

Project: github.com/Ameer3716/EmergencyNavigation
Guide checked: 23 September 2026

Set up WSL on Windows

1 Open PowerShell as Administrator

On Windows, open Start, search for **PowerShell**, right-click it, and choose **Run as administrator**. Run the next two commands. If Windows asks you to restart, restart before continuing.

COPY INTO ADMINISTRATOR POWERSHELL

```
wsl --install  
wsl --update
```

IF WSL IS ALREADY INSTALLED

Running these commands again is usually harmless. You can check with `wsl --status`.

2 Import the official **opp_env** WSL image

This image already includes the Linux tools needed by **opp_env**. The command downloads it and creates a WSL distribution named **opp_env**. The download may take a while.

COPY THIS ONE-LINE COMMAND INTO POWERSHELL

```
curl.exe -L https://github.com/omnetpp/opp_env/releases/download/wsl/opp_env.wsl | wsl --import opp_env -
```

3 Check that WSL created **opp_env**

COPY INTO POWERSHELL

```
wsl -l -v  
wsl -d opp_env --cd ~
```

The first command should list **opp_env** with WSL version **2**. The second opens a Linux terminal. Leave this Linux terminal open for the next pages.

PROMPT CHECK

You are in the correct place when the prompt belongs to **opp_env** and Linux commands such as `pwd` work. Do not run the remaining steps in PowerShell.

Install the four simulation packages

4 Create the framework workspace

Run this once in the **opp_env** Linux terminal. A workspace is simply the folder that holds OMNeT++, Veins, and INET.

COPY INTO WSL (BASH)

```
mkdir -p /home/opp_env/workspace
cd /home/opp_env/workspace
opp_env init
```

5 Install OMNeT++, Veins, and INET

This one command asks **opp_env** to download and build the three exact framework versions. It may take a long time and use several gigabytes. Wait for it to finish; keep the terminal open.

COPY INTO WSL (BASH)

```
opp_env install veins-5.3.1 inet-4.6.0 omnetpp-6.3.0 --no-pause
```

WHY INET IS HERE

INET is installed to match the verified framework environment. The project's active vehicle radio model comes from Veins.

6 Install exact SUMO 1.18.0

The SUMO version bundled with the general framework environment may be different. This project uses its own copy of SUMO 1.18.0 at a fixed path.

COPY INTO WSL (BASH)

```
python3 -m pip install --target /home/opp_env/sumo118_pkg \
  'eclipse-sumo==1.18.0'
/home/opp_env/sumo118_pkg/sumo/bin/sumo --version
```

The version command must show **1.18.0**. If it says a different version, use the full path above instead of a plain sumo command.

MISSING SHARED LIBRARY?

If SUMO reports a missing X11/OpenGL library, run `sudo apt-get update` and then `sudo apt-get install -y libx11-6 libxext6 libxrender1 libgl1`.

Get and build the project

7 Clone the project into the Linux filesystem

Stay inside **opp_env**. The Linux home folder is recommended for faster builds. The command creates an **EmergencyNavigation** folder automatically.

COPY INTO WSL (BASH)

```
cd /home/opp_env/workspace
git clone https://github.com/Ameer3716/EmergencyNavigation.git
cd EmergencyNavigation
```

8 Enter the complete framework shell

This shell supplies the compiler, **make**, OMNeT++ tools, and libraries. Use it again whenever you open a new WSL terminal.

COPY INTO WSL (BASH)

```
cd /home/opp_env/workspace
opp_env shell veins-5.3.1 inet-4.6.0 omnetpp-6.3.0
export VEINS_ROOT=/home/opp_env/workspace/veins-5.3.1
export SUMO_HOME=/home/opp_env/sumo118_pkg/sumo
cd /home/opp_env/workspace/EmergencyNavigation
opp_run --version
```

The last command should report **OMNeT++ 6.3.0**. If **make** cannot be found later, you are probably outside this framework shell.

9 Build the project's custom library

Do this once after cloning, and again after any source-code change.

COPY INTO WSL (BASH)

```
bash scripts/build.sh
find src/out -name libsrc.so -print
```

The second command should print a path such as `src/out/clang-release/libsrc.so` or `src/out/gcc-release/libsrc.so`.

FRESH CLONE TIP

Do not start with **make** in the **src** folder. The build script generates the Makefile first. You can use `bash scripts/...` even if a script's executable bit was lost during copying.

Run your first simulation

10 Start the Veins-to-SUMO launch daemon

This small background service waits on port **9998** and starts SUMO when OMNeT++ needs it. Run it from the project root in the framework shell.

COPY INTO WSL (BASH)

```
mkdir -p artifacts/logs
python3 "$VEINS_ROOT/bin/veins_launchd" -d -p 9998 -vv \
  -c "$SUMO_HOME/bin/sumo" \
  -L "$PWD/artifacts/logs/grid-launchd.log"
```

11 Run the small Smoke test

Smoke uses only one configuration and ends at simulation time **120 seconds**. It is the quickest end-to-end check before a batch.

COPY INTO WSL (BASH)

```
bash scripts/run_grid.sh Smoke
tail -n 5 artifacts/logs/grid-Smoke-stdout.txt
ls -lh simulations/grid/results/Smoke-#0.*
```

Success looks like **End.** in the log and three files with extensions **.sca**, **.vec**, and **.vci**. The run itself may take seconds or minutes depending on the computer.

Watch the simulation on screen

The **Smoke** test on the previous page runs without windows. A graphical view is useful for understanding the roads and vehicles, but it is not needed for the batch.

12

Optional: watch SUMO-GUI

Use this for one run when your Windows WSL installation supports Linux GUI windows. This replaces the terminal SUMO daemon and then runs Smoke.

COPY INTO WSL (BASH)

```
python3 "$VEINS_ROOT/bin/veins_launchd" -d -k -p 9998 -vv -s \
  -c "$SUMO_HOME/bin/sumo-gui --start {}" \
  -L "$PWD/artifacts/logs/grid-launchd-gui.log"
bash scripts/run_grid.sh Smoke
```

To see the **OMNeT++ Qtenv window** as well, keep the GUI daemon above but run the next command **instead of** `bash scripts/run_grid.sh Smoke`. Click **Run** in Qtenv; SUMO-GUI opens when traffic starts.

OPTIONAL: OPEN OMNET++ QTENV

```
cd simulations/grid
library="$(find ../../src/out -name libsrc.so -print -quit)"
test -n "$library"
opp_run -u Qtenv -c Smoke \
  -n "$VEINS_ROOT/src/veins:../../src" \
  -l "$VEINS_ROOT/src/veins" \
  -l "${library%/libsrc.so}/src" -f omnetpp.ini
cd ../../
```

BEFORE A LARGE BATCH

Switch back to command-line SUMO using the restore command on page 7. Otherwise the batch may open a new GUI window for every run.

Run the batch experiment

13 Restore the headless daemon if you used the GUI

Skip this step if you did not run SUMO-GUI. The `-k` option replaces the prior daemon.

COPY INTO WSL (BASH)

```
python3 "$VEINS_ROOT/bin/veins_launchd" -d -k -p 9998 -vv \
  -c "$SUMO_HOME/bin/sumo" \
  -L "$PWD/artifacts/logs/grid-launchd.log"
```

14 Try one matched seed across all five configurations

This verifies the same traffic seed with all five routing/preemption settings before spending hours on the full experiment. The wrapper starts the daemon itself if none is listening.

COPY INTO WSL (BASH)

```
bash scripts/run_batch_env.sh \
  --configs FogCloudAStar MistAStar MistDynamicAStar \
  MistDynamicFogFallback NoPreemptionBaseline \
  --densities low --seed-start 1 --seed-end 1
```

CHECK THE FILES

```
ls -lh results/raw/*-low-seed1.{sca,vec,vci}
ls -lh artifacts/logs/batch/*-low-seed1-stdout.txt
```

You should see **15 raw files**: 5 configurations times 3 file types. There should also be 5 stdout logs.

15 Run the complete 450-case batch

There are 5 configurations, 3 traffic densities, and 30 matched seeds. This can take hours and consume several gigabytes of disk space.

COPY INTO WSL (BASH)

```
bash scripts/run_batch_env.sh \
  --configs FogCloudAStar MistAStar MistDynamicAStar \
  MistDynamicFogFallback NoPreemptionBaseline \
  --densities low medium high --seed-start 1 --seed-end 30
```

IF INTERRUPTED

Run the same batch command again. Completed cases with nonempty `.sca`, `.vec`, and `.vci` files are skipped automatically. Do not add `--force` unless you intend to rerun and overwrite selected results.

ONE-COMMAND START/STOP CHECK - 23 SEP 2026

Copy all lines of Step 15 as one command. In a temporary checkout using the existing WSL tools and built library, the first case finished at simulation time 900 s and wrote `.sca`, `.vec`, and `.vci` files. Ctrl+C stopped the next case. This was **not** a new 450-case run or fresh installation.

Process and check the results

16 Leave the simulation shell

After all 450 runs complete, type **exit** once to leave the **opp_env shell**. You remain in the WSL Linux terminal.

COPY INTO WSL (BASH)

```
exit
cd /home/opp_env/workspace/EmergencyNavigation
```

17 Install the Python analysis packages

The tested WSL image includes **uv**. Use it to create a project-only Python environment; the image's system Python may not support `python3 -m venv`.

COPY INTO WSL (BASH)

```
uv venv .venv
uv pip install --python .venv/bin/python -r requirements.txt
source .venv/bin/activate
```

18 Create tables, graphs, and checks

Run these in order from the project root. The first command's `--require-all` option reports missing cases instead of silently processing an incomplete batch.

COPY INTO WSL (BASH)

```
python analysis/process_results.py --batch --require-all
python scripts/generate_final_graph.py
python analysis/extract_fallback_evidence.py
python scripts/verify_response_times.py
python scripts/audit_batch.py
python scripts/generate_checksums.py
```

19 Find the finished files

INSPECT THE OUTPUTS

```
ls -lh results/processed/summary-batch.csv
ls -lh results/processed/individual_runs-batch.csv
ls -lh results/graphs/ | head
ls -lh artifacts/checksums-local.sha256
```

ORIGINAL EVIDENCE IS NOT IN GITHUB

The repository does not include the earlier 450-run raw files, graphs, or screenshots. A fresh clone creates its own evidence by running the batch. The committed historical checksum file cannot be fully checked against source alone.

If something goes wrong

WHAT YOU SEE	WHAT TO DO
make: command not found	Enter <code>opp_env shell veins-5.3.1 inet-4.6.0 omnetpp-6.3.0</code> from the workspace, then repeat the exports on page 4.
Missing libsrc.so	Run <code>bash scripts/build.sh</code> from the project root.
Port 9998 already in use	A Veins daemon may already be running. Use the page-7 restore command with <code>-k</code> before a batch.
SUMO-GUI waits at time 0	Use the GUI launch command on page 6, which includes <code>--start {}</code> .
No GUI window	Check <code>echo \$DISPLAY</code> in WSL; update WSL with <code>wsl --update</code> in PowerShell.
Batch stops partway	Read the latest <code>artifacts/logs/batch/*-stdout.txt</code> , fix the error, then rerun the same batch command.

What was verified

On the project's WSL setup, the custom library built, Smoke reached its 120-second limit, SUMO-GUI-backed Smoke completed, and one matched low-density seed completed for all five configurations. Fresh Windows/WSL installation and a new 450-case batch were not rerun while preparing this guide.

Official references

Microsoft WSL setup - <https://learn.microsoft.com/en-us/windows/wsl/install>

Official opp_env WSL installation - https://github.com/omnetpp/opp_env/blob/main/INSTALL.md

Veins 5.3.1 compatibility - <https://veins.car2x.org/download/>

INET versions for OMNeT++ 6.3 - <https://inet.omnetpp.org/Download.html>

SUMO 1.18.0 package - <https://pypi.org/project/eclipse-sumo/1.18.0/>

EmergencyNavigation source - <https://github.com/Ameer3716/EmergencyNavigation>